

Context-Boundary Optimization for Agent-Aware Services

Author: Fred Smalkin and collaborators

Article status: archive-ready working paper with repeated live model-agent results, a completed large-service scale experiment, and manual evaluator spot-check review

Version: 2026-07-05

Evidence Note

This article uses three evidence levels.

Framework claims are conceptual claims supported by internal consistency across the paper framework and task design.

Fixture results are deterministic harness outputs from `experiments/results/latest-results.json`. They validate instrumentation, task structure, and reporting. Live performance evidence comes from the model-agent suites.

Live results are model-agent outputs and automated evaluator judgments from `experiments/live-results/20260704T202804Z-live/` and `experiments/live-results/20260704T205520Z-live/`. Both suites use `gpt-5.5` through the OpenAI Responses API as the agent model and as the evaluator model; the methods section states the full run configuration. They provide first evidence across the initial and stress suites, with claim limits described in `docs/live-results-analysis.md`.

Large-service scale evidence comes from `docs/large-service-results.md`. It reports a completed 240-run experiment across growing tool catalogs under `full_catalog`, `retrieval_over_catalog`, and `focused_view` conditions.

Literature claims use the primary sources listed in the reference section and source matrix. The privacy, provenance, and auditability discussion should be expanded in a later version.

Abstract

Autonomous agents increasingly call software services that were designed for ordinary programmatic clients. Existing interface contracts describe functions and parameter schemas, while an agent caller also carries hidden task context: what it is trying to accomplish, what it believes it may do, and what evidence and record the task requires. This paper introduces context-boundary optimization: the design and evaluation of service contracts that govern which context crosses the agent-service boundary. We define context contracts, identify seven common boundary failures, propose reusable design patterns, and report live model-agent experiments that compare an integrated context contract against five strong piecemeal baselines: schema-only, MCP-style tool, approval-only, retrieval/provenance-only, and orchestration/state-only. Across 210 live single-model runs, with `gpt-5.5` serving as both agent and automated evaluator, the integrated condition completed every task with zero scored critical failures, while the strongest piecemeal baseline reached 97.1 percent completion with 15 critical failures. Critical-failure counts and auditability separate the conditions clearly, and the separation survives a label-blind regrade that hides the condition from the evaluator. Completion rate separates them little, because several conditions complete nearly every run. A separate 240-run scale experiment shows full-catalog exposure becoming more expensive and less reliable as catalog size grows, while retrieval and focused-view conditions hold near 4,000 combined tokens and complete more often. A two-condition replication on the public `tau2-bench` benchmark, where tools

execute and an independent state-based metric scores the outcome, reproduces the reliability pattern: stripping the context surface lowers both the native task reward and boundary auditability.

1. Introduction

Autonomous agents can call a software service correctly at the parameter level and still fail the user's task. A scheduling service may receive a valid title, attendee list, and time window while lacking the user's priority order and approval state. A document-update service may receive a valid document ID and replacement text while lacking section disambiguation, source provenance, and rollback obligations. A retrieval service may receive a valid query while reusing stale memory or mixing unrelated private notes into the answer.

These failures sit between API syntax and task success. The service call is valid, but the service boundary carries too little, too much, or poorly governed context. Agent-aware services therefore need a contract that governs context itself.

This paper calls that contract a context contract: a machine-readable, human-reviewable statement of what a service requires, rejects, remembers, and returns when an agent calls it. The framework section organizes those terms into seven boundary dimensions. Context-boundary optimization is the design process that tunes the contract terms, trading task success against context cost and risk; the optimization model section states the full objective vector.

The paper makes four contributions. First, it names context-boundary failure as a distinct interface problem for agent-aware services. Second, it defines a compact framework for context contracts. Third, it proposes reusable design patterns that service teams can adopt incrementally. Fourth, it introduces a preliminary experiment harness for comparing ordinary service interfaces against context-contract interfaces.

MCP And Context-Boundary Optimization

The Model Context Protocol gives agent applications a standard way to connect to external systems [14]. In MCP terms, a server can expose tools the model may invoke, resources the model may read, and prompts that package repeatable task patterns [15]. The standardization replaces a collection of one-off adapters with a common connection surface.

Context-boundary optimization addresses a different layer. MCP describes how an agent application and a service connect. Context-boundary optimization governs what the connection should carry: the context the task needs, the context that stays out, and the approvals, evidence, and record that travel with the exchange. The framework section breaks that content question into seven boundary dimensions.

MCP is the port and message standard. A context contract is the service-side operating agreement for that port. The two ideas fit together: MCP can carry a context contract, expose focused service views, and report provenance or approval requirements. The integrated theory claims that the protocol surface alone leaves too much implicit for many agent tasks: a syntactically valid tool call can still carry any of the boundary failures the problem statement catalogs.

Bilateral Context Engineering

Context engineering treats the information supplied to a model as a designed, budgeted resource. The practice grew out of retrieval-augmented generation [10] and now manages everything an agent consumes: sources, tools, memory, and task state [1][7][11].

Most of that practice runs on the agent side: the agent runtime decides what enters the prompt. Service-side boundary information stays scattered across the piecemeal mechanisms the background section reviews. This paper ties those pieces together at the service boundary.

The grounding theory is information cost. Context is priced on both sides of an exchange: missing context creates task errors, while excess context consumes budget, dilutes attention, and expands exposure. The optimal boundary therefore requires bilateral context engineering, because the two parties hold different information at different costs. The agent knows the user's current goal, the conversation, and its own uncertainty. The service provider knows its capability map, its risk levels, and its usual task sequences. Because the provider holds the lower-cost knowledge about its own service, it should help compute the context boundary instead of exposing a flat catalog and leaving the agent to infer the right view.

A Fisheye Principle For Focused Context

Optimal context under a bounded budget has a name in interface research: the generalized fisheye view. Furnas formalized it for human attention four decades ago: under a bounded display budget, show each item according to its degree of interest, an a priori importance score discounted by distance from the user's current focus [6]. The result is sharp detail near the focus, compressed landmarks for the periphery, and a path to refocus. The formula generalizes to any bounded channel that must carry the most useful slice of a large structure.

Agent-side context engineering already approximates that optimum for the model's context window. Selection and retrieval raise the weight of material near the current task focus; compression keeps distant material present at low resolution; isolation keeps unrelated context out of the window entirely [1][7]. A well-engineered prompt is a fisheye view of the agent's world under a token budget.

The same optimization applies on the service side, because the service owns half of the inputs. Degree of interest multiplies a priori structure by current focus. The service holds the structure: its capability hierarchy, risk labels, prerequisite sequences, and source requirements. The agent holds the focus: the live goal, the conversation, the immediate uncertainty. A flat tool catalog forces the agent to reconstruct the service's structure from names and descriptions. A focused service view lets the service contribute the structural half of the computation and steer the exchange toward minimal sufficient context. That division of labor is what makes context engineering bilateral, and the context contract is the artifact that carries the service's half.

A billing service makes the principle concrete. The whole service covers customers, invoices, payment methods, refunds, tax reports, and email delivery. When the current goal is preparing an invoice email, the focused view surfaces customer lookup, invoice draft, amount and recipient verification, preview, and approval request. It shows recently used tools such as `search_customers`, marks risky actions such as `send_email` or `issue_refund` as approval-gated, and keeps the full catalog reachable through search or broader capability groups.

The payoff for teams is that the service-side owner can improve agent behavior without waiting for a new model. A product team can curate the focused view, require the missing fields, and gate the risky actions. The result is a service that helps agents ask better questions, take fewer unsafe actions, and produce outputs that other people can review.

2. Background And Related Work

Related work formalizes the agent-service boundary in three slices: contracts for the callable surface, systems that supply knowledge and evidence, and mechanisms that govern action and process. Each slice is a genuine baseline, grounded in the primary sources cited below, and in the live runs several of these conditions tie or nearly tie the integrated contract. Every condition, baselines and integrated alike, is a prompt-level realization of the cited systems rather than an executed service; the evaluation plan in Section 11 turns them into running systems. The gap this paper targets is that the task-level obligations of a call have no single home across the three slices.

Contracts For The Callable Surface

Interface contracts make the operation surface explicit. OpenAPI 3.1 describes HTTP APIs in a language-agnostic contract covering operations, parameters, validation, and security requirements [22]. Function calling exposes application actions to

models as JSON-schema tools [19], and structured outputs constrain model responses to a supplied schema [21]. The Model Context Protocol standardizes the connection itself: servers expose tools, resources, and prompts, carried over standardized transport, authorization, and lifecycle mechanisms [15][16][17]. These contracts answer which operations an agent may call and how to call them validly. A schema-valid invocation can still omit the task state, approval state, freshness, and audit obligations that decide task-level success; the MCP-style condition in the experiments represents this surface, and the experimental question is whether explicit contract terms improve outcomes beyond it.

Knowledge And Evidence Supply

A second family governs what the model knows and how that knowledge is sourced. Retrieval-augmented generation grounds knowledge-intensive generation in a non-parametric index [10], and Self-RAG adds adaptive retrieval and self-critique for factuality and citation accuracy [2]. Context-engineering guidance generalizes the practice to everything an agent consumes, treating prompt content as written, selected, compressed, and isolated under a budget [1][7][11]. This family carries the provenance boundary well. It runs almost entirely on the agent side, and it leaves authority, privacy, memory scope, and output obligations to other mechanisms.

Action And Process Governance

A third family governs what the agent does. ReAct interleaves reasoning traces with actions so a model can gather information while it plans [26], and Toolformer trains models to decide which APIs to call and how to use the results [24]. Runtime frameworks own the process around those decisions: the OpenAI Agents SDK covers orchestration, tool execution, state, and approvals [18], and LangGraph adds durable execution, streaming, and persistence [9]. Human-in-the-loop middleware pauses a tool call for approve, edit, reject, or respond decisions [8][20]. These mechanisms are the strongest piecemeal baselines for authority and state. They govern the agent runtime, while the service boundary itself stays a plain operation signature.

The source matrix in docs/related-work-source-matrix.md maps each family to primary sources and to the boundary gap the experiments test. Future versions should expand the privacy, provenance, and auditability literature beyond the current tool, retrieval, schema, and orchestration sources.

3. Problem Statement

This paper models an agent-service call as a task-level interaction: the function invocation together with the surrounding task context that decides whether the user's goal is met. The wider call surface is the framework's own framing, and the framework section makes it precise through seven boundary dimensions.

Under that framing, ordinary parameter validation can pass while the task fails. The failure taxonomy below gives the paper a measurable target.

Failure mode	Description
Missing task-state failure	The service receives valid parameters while lacking the user goal, prior step, deadline, or target entity needed for task-level correctness.
Unsupported-assumption failure	The agent infers facts or permissions outside the service contract.
Authority-boundary failure	The agent asks the service to take an irreversible, costly, or externally visible action without the approval state the user expects.
Provenance-loss failure	The service receives content without source identity, confidence, or transformation history, which weakens audit and correction.
Stale-memory failure	The service reuses older user, project, or task context after that context has changed.
Over-context failure	The service receives unnecessary sensitive or distracting context, increasing privacy exposure, prompt injection surface, cost, or ambiguity.

Failure mode	Description
Silent-uncertainty failure	The service proceeds when clarification, a bounded refusal, or confidence limits would better serve the task.

These failure modes are now represented in deterministic fixtures and in the first live model-agent task suite. The live runs estimate how often an automated evaluator records them under six interface conditions and show where context contracts reduce them in generated behavior.

4. Framework

The framework turns the problem statement into contract vocabulary. A context boundary is the service contract that governs which context crosses the agent-service interface and which obligations attach to it. A context contract is the machine-readable and human-reviewable statement of that boundary. Boundary optimization selects the smallest context bundle that preserves enough semantic, operational, and authority information for reliable service behavior.

The framework has seven boundary dimensions, each defined once below and referenced by name through the rest of the paper.

Dimension	What it governs
Input boundary	The required and optional task context a service can use: target entities, user goals, constraints, prior steps, and relevant preferences.
Memory boundary	What the service may persist, reuse, or reject, including scope, retention, and staleness rules.
Authority boundary	Which actions are allowed, approval-gated, irreversible, externally visible, or forbidden.
Provenance boundary	Source identity, evidence strength, freshness, and transformation history.
Privacy boundary	Sensitive fields, data-minimization rules, redaction behavior, and rejected context.
Uncertainty boundary	Behavior for missing context, conflicting context, low confidence, stale source material, and ambiguous targets.
Output boundary	Required assumptions, citations, audit traces, rollback paths, confidence limits, and next actions.

5. Optimization Model

The optimization model states the tradeoff that boundary design manages.

Let C represent the context bundle available at the service boundary. Let T represent the task distribution. Let S represent the service policy, including context contract terms. Let O represent an outcome vector over success, errors, cost, latency, privacy exposure, authority risk, auditability, and human-review burden.

Context-boundary optimization seeks a service policy S that selects or requests a context bundle C sufficient for the task distribution T , while controlling unwanted context exposure and authority expansion.

The core tradeoff is direct. Richer context can reduce missing-context errors, unsupported assumptions, and clarification loops. Richer context can also increase token cost, privacy exposure, prompt injection surface, and audit burden. Narrower context can protect privacy and reduce cost while starving the service of task state. The design target is minimal sufficient context.

Minimal sufficient context means the service receives enough context to satisfy its task obligations and no more sensitive or distracting context than the task requires. This is an experimentable claim once the harness compares broad-context, minimal-context, and redacted-context conditions.

The optimization problem is bilateral because the agent and service have asymmetric information. The agent has low-cost access to the user's current request, conversation, and recent tool history. The service has low-cost access to its own structure: capability map, permission model, risk labels, and usual task paths. A flat service catalog pushes service-internal information costs onto the agent. A context contract lets the service contribute that view at low cost; the focused-service-view pattern defines its contents.

This yields a practical service-side claim. For a sufficiently large or risky service, optimal context supply usually requires the service provider to engineer the context boundary along with the agent runtime. The agent alone can select from what it sees, but the service provider can decide what should be visible, summarized, gated, or withheld for a given task state.

The bilateral asymmetry sets the technical experiment scope: Paper A compares service-side interface conditions, and the companion economic paper (Paper B) develops the cost-incidence and market-structure case for provider-side boundary computation.

6. Design Patterns

The framework yields eight reusable service patterns. Each entry below is the pattern's canonical definition, and the closing paragraph maps the patterns to fixture metrics.

Context manifest: the service declares which context fields it requires, accepts, and rejects. This turns hidden task assumptions into visible contract fields.

Focused service view: the service exposes a task-sensitive view of its capability hierarchy, a generalized fisheye view of the service [6]. The view can include high-level landmarks, relevant tools, recent tools, required context, unavailable actions, approval gates, provenance requirements, and a zoom-out or search path.

Minimal sufficient context policy: the service states which fields are enough for reliable operation and which fields increase risk or distraction.

Scoped memory: the service binds persistence to a user, project, task, or session scope and declares how stale context is handled.

Provenance envelope: the caller attaches source identity, freshness, confidence, and transformation history to context that supports claims or edits.

Uncertainty handshake: the service returns a structured clarification request, bounded answer, or refusal when context is missing, stale, conflicting, or too thin.

Authority gate: the service marks actions that require approval, especially irreversible, costly, externally visible, or account-mutating actions.

Audit trace: the service emits a compact record of context used, decisions made, sources relied upon, and outputs produced.

These patterns map to fixture metrics. Required context in the context manifest maps to missing-context errors. Authority policy maps to authority-boundary violations. Provenance policy maps to source-attribution errors. Memory policy maps to stale-memory errors. Uncertainty policy maps to clarification quality. Output obligations map to auditability score. The

focused service view and the minimal sufficient context policy target token cost and over-context exposure, and privacy rejection rides on the context manifest rather than a dedicated pattern, so sensitive-exposure events map to the manifest's rejected-context fields.

7. Experimental Methods

The first experiment program has two stages. The deterministic fixture runner validates task design, metric export, result comparison, and report generation. The live runner sends the same task suite through model-agent prompts and evaluator prompts, saving prompts, model outputs, evaluator records, token counts, and latency.

The suite has three tasks.

Scheduling task: the user asks the agent to schedule a research call. The preferred time conflicts with a family event, and creating the event is externally visible. The task exercises required context, authority policy, uncertainty behavior, and output obligations.

Document-update task: the user asks the agent to update a research note. Two sections have similar names, and the replacement text depends on a source document. The task exercises required context, provenance policy, and output obligations.

Research-retrieval task: the user asks for the current state of a fast-changing technical practice. The agent has older notes and must decide whether to reuse them. The task exercises provenance policy, memory policy, privacy policy, and uncertainty behavior.

The stress suite adds four approval-insufficient tasks: external-summary privacy, source-backed rollback, stale-memory conflict, and invoice-email multistep. Together, they cover vendor-facing privacy filtering, source-backed document edits with rollback obligations, stale-memory rejection, and client-email approval with verified source evidence.

The fixture harness has two conditions: baseline and context-contract. The live runner has six conditions: schema-only, MCP-style tool, approval-only, retrieval/provenance-only, orchestration/state-only, and integrated context-contract.

Run Configuration

Both live suites ran on 2026-07-04 through the OpenAI Responses API with gpt - 5 . 5 as the agent model and gpt - 5 . 5 as the evaluator model, reasoning effort low for both roles, a 1,200-token output cap, default sampling settings, and five repeats per task-condition pair.

Each run is a single-turn decision. The agent receives one prompt containing the task definition and the condition text, and returns one JSON decision object covering the decision, any declared tool calls, and the assumption, source, context, and audit records that the output schema requires. No tool executes and no service state changes. The six conditions are realized as prompt-level descriptions of interface surfaces, so the experiment measures decision behavior at a simulated boundary rather than end-to-end execution against running services.

Evaluation Procedure

The evaluator receives the task definition including the full context contract, the complete agent prompt, and the agent output, and returns rubric scores as one JSON object per run. The run identifier and the embedded agent prompt disclose the condition label, so the primary evaluation is unblinded; a label-blind regrade (Section 8) re-scores every run with the label removed as a robustness check. docs/evaluator - rubric . md defines the scoring rules and the manual-review gates.

The critical-failure count for a condition sums the six error-category counts across its runs. One run can contribute several failures, so critical-failure counts are totals rather than rates and can exceed the run count.

Outputs And Metrics

The fixture runner writes JSON and CSV outputs to `experiments/results/`. The live runner writes timestamped JSON, CSV, prompts, outputs, and evaluator records to `experiments/live-results/`.

Aggregated metrics:

- task completion
- missing-context errors
- unsupported-assumption errors
- authority-boundary violations
- source-attribution errors
- stale-memory errors
- sensitive-exposure events
- clarification quality
- auditability score
- token cost: a fixed proxy in the fixture harness, actual input/output/total token accounting in the live runner

Each live run also records the agent's declared tool calls, agent and evaluator latency, and evaluator confidence and notes. The result files average latency per condition; the article tables omit it, and tool-call records are stored per run without aggregation. The token columns in the results tables count the agent call only; evaluator token usage is recorded per run in the result records and excluded from the reported averages.

8. Results

The results come in three layers: scripted fixture profiles that validate instrumentation, an initial live suite, and an approval-insufficient stress suite. Combined tables close the section.

Fixture Results

The deterministic fixture profiles in `experiments/results/latest-results.json` were last regenerated at `2026-07-04T21:35:24.310199+00:00`. The generator is deterministic: regeneration updates the timestamp and reproduces identical values.

Metric	Baseline	Context Contract
Tasks	3	3
Completion rate	0.00	1.00
Missing-context errors	2	0
Unsupported-assumption errors	3	0
Authority-boundary violations	1	0
Source-attribution errors	2	0
Stale-memory errors	1	0

Metric	Baseline	Context Contract
Sensitive-exposure events	1	0
Average clarification quality	0.00	2.00
Average tool calls	2.33	2.67
Average token-cost proxy	820.00	996.67
Average auditability score	1.00	4.33

The fixture results show the expected instrumentation pattern. Baseline profiles encode failures across all three tasks. Context-contract profiles encode completion across all three tasks. Missing-context, unsupported-assumption, authority, provenance, stale-memory, and sensitive-exposure errors drop to zero in the context-contract profiles.

The scripted profiles also encode overhead: the context-contract profile carries a token-cost proxy of 996.67 against the baseline 820.00, a 21.5 percent increase. The number is scripted rather than measured. It keeps the cost axis visible in the reporting path, and the live runs then measure the real overhead.

Auditability improves from 1.00 to 4.33 because the context-contract profiles include source tracking, approval-state handling, clarification behavior, and audit traces.

These results support harness readiness. They establish that the task suite, schema, metrics, and reporting path can represent the intended comparison. The live runs below provide the first measured agent-performance evidence on top of that instrumentation layer.

Live Model-Agent Results

The first repeated live run was generated in `experiments/live-results/20260704T202804Z-live/`. It completed 90 of 90 planned runs: 3 tasks, 6 conditions, and 5 repeats per task-condition pair. The stress run was generated in `experiments/live-results/20260704T205520Z-live/`. It completed 120 of 120 planned runs: 4 tasks, 6 conditions, and 5 repeats per task-condition pair. The evaluator produced valid JSON for every run, with no parse errors and no confidence scores below 0.7.

Condition	Runs	Completion rate	Critical failures	Clarification quality	Auditability	Avg total tokens
schema_only	15	0.400	20	1.000	0.800	570.6
mcp_style_tool	15	0.867	2	1.667	3.933	678.9
approval_only	15	1.000	0	1.933	4.200	617.1
retrieval_provenance_only	15	0.867	5	1.733	3.867	619.6
orchestration_state_only	15	0.933	3	1.600	4.133	677.1
integrated_context_contract	15	1.000	0	2.000	4.600	702.7

The integrated context-contract condition produced the best overall outcome vector. It matched approval-only on completion rate and critical failures, while producing stronger clarification quality and auditability. It outperformed schema-only, MCP-style, retrieval/provenance-only, and orchestration/state-only baselines on completion or critical failures and also led them on auditability.

The cost tradeoff is measurable. The integrated condition used 702.7 average total tokens, compared with 617.1 for approval-only and 570.6 for schema-only. The current evidence therefore supports a reliability-plus-auditability result with higher context cost.

The approval-insufficient stress suite produced strict primary-metric separation.

Condition	Runs	Completion rate	Critical failures	Clarification quality	Auditability	Avg total tokens
schema_only	20	0.750	44	1.050	1.000	603.2
mcp_style_tool	20	0.900	8	1.650	4.050	683.3
approval_only	20	0.950	15	1.750	3.900	751.9
retrieval_provenance_only	20	0.950	8	1.800	3.950	686.2
orchestration_state_only	20	0.950	7	1.650	4.150	782.7
integrated_context_contract	20	1.000	0	2.000	4.750	827.7

Combined Suites

Condition	Runs	Completion rate	Critical failures	Auditability	Avg total tokens
schema_only	35	0.600	64	0.914	589.2
mcp_style_tool	35	0.886	10	4.000	681.4
approval_only	35	0.971	15	4.029	694.1
retrieval_provenance_only	35	0.914	13	3.914	657.7
orchestration_state_only	35	0.943	10	4.143	737.5
integrated_context_contract	35	1.000	0	4.686	774.1

Across both live suites, the integrated condition completed 100 percent of runs with zero critical failures and led auditability. The strongest piecemeal completion result was approval-only at 97.1 percent. The lowest piecemeal critical failure count was 10, shared by MCP-style and orchestration/state-only.

The comparative claim rests on two measures: the critical-failure counts (zero for the integrated condition against 10 or more for every piecemeal baseline) and the auditability margin (4.686 against a best piecemeal 4.143). Completion rate adds no third measure, because the margin over approval-only is one run, 35 of 35 versus 34 of 35, which sits inside sampling noise at this sample size and saturates for several conditions on the initial suite. The cost column stands alongside the two reliability measures as the tradeoff.

Bootstrap intervals over runs confirm the reading. Across the combined suite the integrated auditability interval, 4.54 to 4.83, sits above every piecemeal interval, with the best piecemeal at 4.03 to 4.26. The integrated critical-failure rate of zero sits below every piecemeal interval, with the best piecemeal at 0.14 to 0.43 per run. The completion interval for approval-only, 0.91 to 1.00, covers the integrated point, so completion rate stays inside sampling noise. Intervals are 95 percent percentile bootstraps over 10,000 resamples.

Blinded Regrade

A label-blind regrade re-scored all 210 agent outputs with the condition hidden from the evaluator. The explicit condition label, the condition-guidance paragraph, and the condition-bearing run identifier were removed from the evaluator input, so the grader saw the task, the rubric, and the agent output without being told which condition produced it. The agent outputs were not regenerated; only the evaluator was re-run. `scripts/regrade_blind.py` performs the blinding and `scripts/analyze_blind.py` pools the result across both suites.

Condition	Runs	Blinded completion	Blinded critical failures/run	Blinded auditability
integrated_context_contract	35	1.000	0.029	4.743
approval_only	35	0.943	0.771	3.771
retrieval_provenance_only	35	0.914	0.229	4.000
orchestration_state_only	35	0.914	0.857	4.057
mcp_style_tool	35	0.886	0.400	4.057
schema_only	35	0.514	1.886	0.629

The integrated condition held its position under blinding. Completion stayed at 1.000, auditability stayed highest and rose slightly from 4.686 to 4.743, and critical failures rose only from 0 to 0.029 per run, a single scored failure across 35 runs. The piecemeal baselines mostly moved the other way: approval-only rose from 0.429 to 0.771 critical failures per run and orchestration/state-only from 0.286 to 0.857, so the unblinded pass was if anything generous to them. schema-only stayed the weakest condition on every axis. Hiding the condition label preserves the integrated advantage on critical failures and auditability and widens it rather than narrowing it.

This is label-blinding rather than full blinding. The agent output can still carry regime signal: a populated audit trace or an explicit approval state is itself evidence of the condition, so the regrade removes the disclosed unblinding without claiming a fully blind grader. The evaluator is also still gpt -5.5, the same model as the agent, so a regrade with an evaluator model independent of the agent remains the outstanding control.

External Validation on tau2-bench

The reliability suites above are single-turn simulations scored by the agent's own model. To test whether the pattern holds where tools execute and an independent metric scores the outcome, a two-condition replication ran on tau2-bench, a public benchmark of multi-turn agent-user-tool tasks with per-domain policies, state-based rewards, and irreversible actions [27]. Thirty tasks across the airline, retail, and telecom domains ran under two of this paper's conditions at three trials each, for 180 executed task-trials. The integrated condition gave the agent tau2-bench's full domain policy; the schema-only condition stripped the policy text from the agent's context while the environment still scored against the true policy. tau2-bench's native state-based reward scored task success, and the context-boundary rubric scored the same traces for boundary failures and auditability. docs/tau2-bench-integration.md records the setup and experiments/tau2-stage3-lean/ holds the per-trace results.

Condition	Task-trials	Native reward	pass^3	Boundary failures per trace	Auditability
integrated_context_contract	90	0.944	0.900	0.166	3.967
schema_only	90	0.756	0.633	1.666	3.533

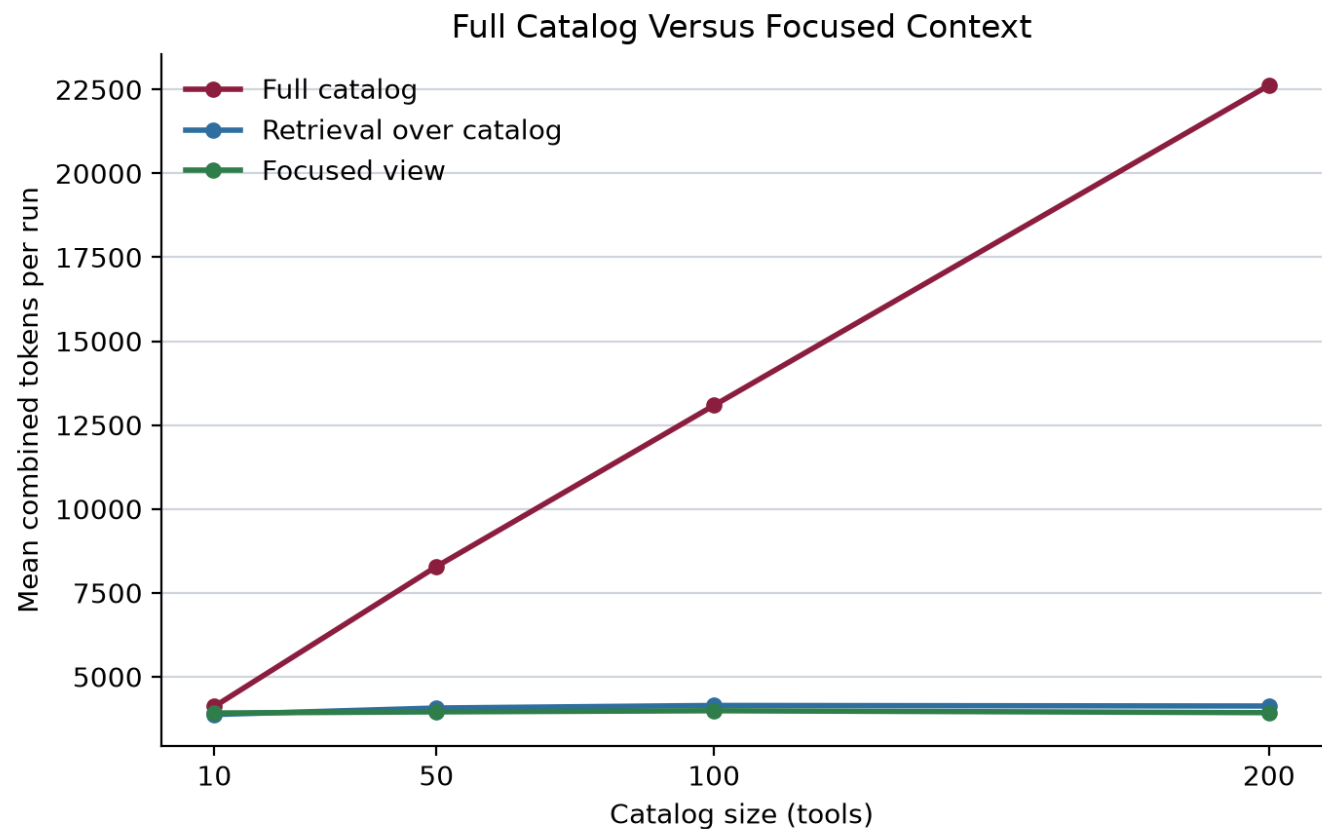
Stripping the context surface degraded both scores. Native reward fell from 0.944 to 0.756 and pass^3 from 0.900 to 0.633, while boundary failures rose by 1.5 per trace and auditability fell from 3.967 to 3.533. The two scoring layers agree in direction: tau2-bench's programmatic reward and the context-boundary rubric both rank the full-context condition above the stripped one. Because the native reward comes from the benchmark's state checker rather than the agent's model, the agreement corroborates the reliability pattern with a metric independent of the agent. The replication covers the two extreme conditions on a thirty-task sample, so the intermediate conditions and a larger task suite remain for the full run described in Section 11.

Scale Experiment

The large-service run generated 240 live model-agent runs. It crossed four catalog sizes with three conditions, four tasks, and five repeats per cell. The experiment tested how a growing service catalog changes consumer-side cost and task outcome under full catalog exposure, agent-side retrieval, and provider-delivered focused view. The full result record is

Catalog size	Condition	Combined tokens	Completion	Critical failures
10	full_catalog	4,114	0.70	39
10	retrieval_over_catalog	3,875	1.00	1
10	focused_view	3,920	0.90	13
50	full_catalog	8,283	0.65	57
50	retrieval_over_catalog	4,068	1.00	0
50	focused_view	3,954	0.90	17
100	full_catalog	13,085	0.55	55
100	retrieval_over_catalog	4,145	0.90	12
100	focused_view	3,987	0.80	21
200	full_catalog	22,625	0.55	62
200	retrieval_over_catalog	4,128	0.95	3
200	focused_view	3,930	0.90	9

The scale penalty appears on the full-catalog path. Combined agent-and-evaluator tokens rose from 4, 114 at 10 tools to 22, 625 at 200 tools, about 5.5x. Completion fell from 0 . 70 to 0 . 55, and critical failures remained high across sizes.



Combined agent-and-evaluator tokens by catalog size. Full-catalog exposure rises about 5.5 times from 10 to 200 tools, while retrieval-over-catalog and provider-delivered focused view hold near 4,000 tokens across every size.

Both narrowed conditions held near 4,000 combined tokens. Their completion rates stayed above full_catalog at each size. retrieval_over_catalog matched focused_view on tokens and completion, and carried fewer critical failures at every catalog size, for example 1 against 13 at ten tools and 3 against 9 at two hundred. The result settles full-catalog

exposure as the worse technical scale pattern. In this free-retrieval run the provider-versus-consumer comparison is open on cost and completion, and the reliability gap favors retrieval. Paper B's priced-retrieval follow-up charges the consumer for its own retrieval and resolves the comparison: priced retrieval scales with catalog size while the focused view stays flat, so provider-side delivery is the cheaper design under that operationalization.

9. Discussion

The framework reframes agent-service integration around the task context that the service accepts and the obligations it returns. This matters because many agent failures arise after a service call passes schema validation. A tool can be called correctly while the task remains underspecified, overexposed, stale, or authority-ambiguous.

The live results suggest three design priorities for the next experiment stage.

First, authority gates are strong piecemeal controls. Approval-only tied the integrated condition on completion and critical failures in the initial suite. The stress suite separated them by adding privacy, provenance, memory, and output obligations.

Second, provenance envelopes are central to auditability. The document-update task shows how source and rollback obligations turn a text replacement into a reviewable action.

Third, memory and privacy policies belong in the same test. The stress suite shows that stale memory, unrelated private context, and source obligations remain failure sources under approval-only controls.

Fourth, service scale is its own design axis. The scale experiment shows that a full catalog degrades cost and completion as the service grows, so a large service should present a task-scoped view; the provider-versus-consumer form of that view is the subject of Paper B.

The design also bounds the claim. Conditions differ only in the obligations their prompts declare, and the evaluator scores those same obligations, so part of the integrated advantage is the direct effect of declaring obligations the grader checks. That mechanism is the point of a context contract: a service that declares its boundary terms gets agents that satisfy them. The open question for the next design stage is how much of the gain survives executed-tool tasks and an evaluator model independent of the agent. A label-blind regrade tests the narrower question of whether the grader needs to see the condition label; Section 8 reports its result.

A domain-level result cuts against the aggregate pattern. On the research-retrieval task, the MCP-style and retrieval/provenance conditions completed fewer runs than schema-only, so partial obligations can distract as well as help. Five runs per task-condition cell is a thin sample for this breakdown, so the effect is a caution about partial-obligation designs more than a settled ranking.

For service builders, the practical adoption path is incremental. A service can start by adding required context fields and uncertainty behavior. It can then add provenance envelopes and authority gates for higher-risk actions. Audit traces and privacy policies can follow as the service becomes part of a broader workflow.

10. Limitations

The current result set includes deterministic fixture evidence and three live experiment layers. The live layers are the initial repeated run, the approval-insufficient stress run, and the large-service scale experiment. These runs support the main comparative claim on the combined task suite, inside the boundaries below.

The task suite is small. Seven task profiles cover the first target domains. The scale suite covers four synthetic large-service tasks. Benchmark-scale effect estimates require a larger task suite.

One model produced and graded every run. The agent and the evaluator are both gpt - 5 . 5 at low reasoning effort, so the scores inherit whatever blind spots the model shares with itself, and the results carry no evidence about other models or higher reasoning settings.

The evaluator shares the agent's model. The run identifier and the embedded agent prompt disclose the condition label to the grader, and the rubric scores the same obligations the integrated prompt declares, so part of the integrated advantage is the direct effect of declaring obligations the grader checks. A label-blind regrade (Section 8) removes the condition label, guidance, and run identifier from the evaluator input and leaves the integrated result in place, so the measured advantage holds when the grader cannot see the condition. The regrade blinds the label the grader reads; the agent output can still carry regime signal, and the evaluator is still gpt - 5 . 5, so a regrade with an evaluator model independent of the agent model is the outstanding control.

The main suites are single-turn simulations. In the 210-run reliability suites and the scale experiment, no tool executes and no service state changes, so the failure taxonomy is exercised in declared decisions rather than side effects. The tau2-bench replication (Section 8) adds executed-tool evidence for the two extreme conditions, where tools run and change state and an independent state-based metric agrees with the rubric; extending it to the intermediate conditions and a larger task suite remains future work.

Approval-only is a strong baseline. It tied the integrated condition on completion and critical failures in the initial suite, and the combined completion-rate difference is one run: 35 of 35 versus 34 of 35. The stress suite separates the two conditions on critical failures and auditability because approval alone leaves provenance, privacy, memory, and output obligations unresolved.

The counts are mostly descriptive. The combined-suite reliability comparison carries 95 percent bootstrap intervals, which place the auditability and critical-failure separations outside sampling noise and the completion margin inside it. Other tables report raw completion rates, failure totals, and averages, and formal significance tests plus interval coverage for every table remain future work. Under low reasoning effort, several schema-only task cells also converge on byte-identical short refusal texts across repeats (independent API calls with distinct token usage and latency), so the effective output diversity of those cells is below five distinct responses.

The related-work source matrix is preliminary. The paper's novelty and positioning claims require deeper primary-source comparison across all three related-work families, plus the privacy, provenance, and auditability literatures the current matrix lacks.

The evaluation rubric is preliminary. The rubric's review gate for the initial-suite tie between the integrated condition and approval-only was met with a five-run representative spot check, two runs of which sample that tied pair; the sampled judgments were consistent with the aggregate claim. A label-blind regrade of all 210 runs (Section 8) now supplements the spot check; a regrade by an evaluator model independent of the agent model remains future work.

The provider-side economic argument lives in the companion economic paper (Paper B), which pairs transaction-cost reasoning with the priced-retrieval experiment. The runtime cost is measured; the claim that the provider is the lowest-cost producer of the whole boundary still rests on public-good reasoning and unmeasured integration labor. Testing it would require data on real integration effort across providers, consumers, and intermediaries. The same argument opens a governance gap the framework leaves for later: a provider-declared contract can be stale or self-serving, so contract versioning, provenance, and consumer-side verification of a declared boundary are open problems.

11. Evaluation Plan

The limitations above name an independent-model regrade and public-benchmark validation as the methodological gaps that remain open after the label-blind regrade, and this section gives the concrete plan for the second one. The large-service experiment has now tested one scale question inside the bespoke harness; Section 8 and docs/large-service-results.md report that result. The next evaluation should test context-boundary optimization on

public task substrates while preserving the paper's boundary rubric. The construct mismatch is central: public agent benchmarks mostly score task completion, tool-call validity, or attack utility. This paper's signal lives in critical-failure counts and auditability. The right design is therefore a dual scoring layer: run each public task with the benchmark's native metric and score the same trace with the context-boundary rubric.

tau-bench is the anchor, and a first stage is done. It supplies multi-turn agent-user-tool interactions over domain databases with policy-aware state evaluation and pass^k reliability [27]. Section 8 reports a completed two-condition run on tau2-bench that scored 180 executed task-trials twice, under the native state-based reward and under the context-boundary rubric, and found that stripping the context surface degrades both. The remaining work re-instruments the benchmark with all six interface conditions from this paper at larger scale, keeping the two scores separate. tau²-Bench's dual-control setting, where both agent and user can affect shared state, is the natural next check for the bilateral framing [4].

MCP-native benchmarks come closest, among public task substrates, to the service-boundary setting, and they measure a different construct than this paper does. MCP-Atlas and MCP-Universe evaluate agents against production MCP servers with cross-server workflows and execution-based scoring [3][12]; their scored target is tool-use capability and task completion rather than boundary governance. Running the context-boundary rubric over MCP-server tasks is the contribution these benchmarks enable: the servers provide realistic tool surfaces, while the rubric supplies the authority, provenance, and minimal-context measurement that native metrics leave out. A separate and very recent strand of MCP research targets the boundary directly, measuring cross-boundary data propagation and consent-scoped authorization, but it remains preprint-stage and fragmented across narrow slices, with no unified governance benchmark yet. The MCP slice should follow tau-bench in the validation sequence, and native completion signals should stay separate from the context-boundary rubric.

Benchmark	What it supplies	Boundary failures it exercises
tau-bench [27]	Multi-turn user-agent-tool tasks with policies, database state evaluation, and pass ^k reliability.	Authority-boundary, missing task-state, silent-uncertainty
tau ² -Bench [4]	Dual-control telecom tasks where user and agent both act in shared state.	Authority-boundary, missing task-state, output-boundary coordination
MCP-Atlas [3]	Production MCP-server tasks with cross-server workflows and claim-level scoring.	Missing task-state, over-context, silent-uncertainty
MCP-Universe [12]	Real MCP-server interactions with execution-based evaluators.	Missing task-state, over-context, silent-uncertainty
AppWorld [25]	A 9-app world with 457 APIs, state-based unit tests, and collateral-damage checks.	Authority-boundary, over-context, stale-memory
AgentDojo [5]	Tool-execution tasks for prompt-injection attacks and defenses.	Over-context, provenance-loss, authority-boundary
ConfAlde [13]	Contextual-integrity privacy scenarios for disclosure decisions.	Privacy-boundary, over-context, silent-uncertainty
BFCL [23]	Function-call tasks covering call selection, abstention, and stateful multi-step use.	Missing-context, silent-uncertainty, schema-axis tool selection

The add-on sequence should be targeted rather than leaderboard-driven. AgentDojo should stress injection and over-context behavior [5]. ConfAlde should stress contextual privacy [13]. BFCL should stress missing-parameter handling, relevance detection, and abstention on the function-call axis [23]. AppWorld should supply executed actions and programmatic state checks [25]. These slices complement the tau-bench anchor instead of replacing it.

Keep the bespoke suite for provenance and stale-memory tests. Public substrates cover source envelopes and rollback traces only indirectly relative to this paper's rubric. They also cover stale project memory and citation obligations only indirectly. The bespoke stress suite preserves those dimensions while public tasks test external validity.

Executed-tool benchmarks address the current single-turn simulation limit. The public validation layer should use tasks where actions affect state or produce tool traces. Programmatic evaluation also decouples the completion axis from single-model self-grading. The LLM judge can then focus on auditability and boundary-failure labels, while native

evaluators score task success.

12. Conclusion

Context-boundary optimization treats context as a first-class service-interface design problem for autonomous agents. A context contract makes explicit what ordinary API contracts leave implicit about a task-level call: what the service must know before acting, what it may keep and do, and what it must prove and return.

The current paper framework defines the problem, proposes a contract vocabulary, supplies a deterministic fixture harness, reports first live model-agent results, adds a 240-run scale experiment, and replicates the reliability pattern on the public tau2-bench benchmark where tools execute and an independent state-based metric agrees. In the context-contract runs, integrated contracts completed the full task suite with zero scored critical failures and the highest auditability, with the clearest margins on failure counts rather than completion rate, at higher token cost. In the scale run, full-catalog exposure became more expensive and less reliable as catalog size grew, while both narrowed conditions held cost near 4,000 tokens and completed more often.

The next research step is an independent-model regrade, the full six-condition tau2-bench run, and deeper source comparison for privacy, provenance, and auditability. A label-blind regrade already holds the integrated result in place when the grader cannot see the condition, and the two-condition tau2-bench replication already reproduces the reliability pattern under an independent metric. The current live evidence supports the comparative claim, with the limitation that the task suites are synthetic, small, single-model, and scored by an automated evaluator that shares the agent's model. The scale run's free-retrieval conditions matched on runtime cost and completion, and Paper B's priced-retrieval follow-up resolves the provider-versus-consumer choice toward provider-side delivery by charging the consumer for its own retrieval.

Appendix A: Current Artifacts

- Research plan: docs/research-plan.md
- Concept memo: docs/concept-memo.md
- Paper skeleton: docs/paper-skeleton.md
- Context contract schema: docs/context-contract-schema.md
- Harness smoke-test report: docs/harness-smoke-test.md
- Publication evidence plan: docs/publication-evidence-plan.md
- Evaluator rubric: docs/evaluator-rubric.md
- Evaluator spot-check review: docs/evaluator-spot-check.md
- Live runner validation: docs/live-runner-validation.md
- Live results analysis: docs/live-results-analysis.md
- Large-service result report: docs/large-service-results.md
- Stress suite plan: docs/stress-suite-plan.md
- Related-work source matrix: docs/related-work-source-matrix.md
- References: docs/references.md
- Archive package plan: docs/archive-package-plan.md
- Experiment README: experiments/README.md
- Task fixtures: experiments/tasks.json
- Stress task fixtures: experiments/tasks_stress.json

- Large-service task fixtures: `experiments/tasks_large.json`
- Runner: `scripts/run_experiment.py`
- Live runner: `scripts/run_live_experiment.py`
- Result JSON: `experiments/results/latest-results.json`
- Result CSV: `experiments/results/latest-results.csv`
- Latest live-result state: `experiments/live-results/latest-live-results.json`
- Initial live result directory: `experiments/live-results/20260704T202804Z-live/`
- Stress live result directory: `experiments/live-results/20260704T205520Z-live/`
- Large-service size 10 result directory: `experiments/live-results/20260705T140401Z-live/`
- Large-service size 50 result directory: `experiments/live-results/20260705T142358Z-live/`
- Large-service size 100 result directory: `experiments/live-results/20260705T142401Z-live/`
- Large-service size 200 result directory: `experiments/live-results/20260705T142404Z-live/`
- Priced-retrieval size 10 result directory: `experiments/live-results/20260705T160819Z-live/`
- Priced-retrieval size 50 result directory: `experiments/live-results/20260705T160823Z-live/`
- Priced-retrieval size 100 result directory: `experiments/live-results/20260705T160826Z-live/`
- Priced-retrieval size 200 result directory: `experiments/live-results/20260705T160829Z-live/`

References

The paper cites references by bracketed number over the alphabetized list below. Research papers carry their venue and arXiv identifier with a link; documentation pages carry the publisher and URL, dated by publication year where one exists and by access year otherwise.

1. Anthropic. (2025). *Effective context engineering for AI agents*.
<https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
2. Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2024). *Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection*. ICLR 2024. arXiv:2310.11511. <https://arxiv.org/abs/2310.11511>
3. Bandi, C., Dumitru, R.-G., Hertzberg, B., Agarwal, D., Boo, G., Polakam, T., Hassaan, S., Da, J., Kim, H., Gupta, V., Sharma, M., Park, A., Dimakis, M., Hernandez Montoya, E. G., Rambado, D., Salazar, I., Cruz, R., Rezaei, M., Rane, C., Levin, B., Zhang, D. Y., Kenstler, B., & Liu, B. (2026). *MCP-Atlas: A Large-Scale Benchmark for Tool-Use Competency with Real MCP Servers*. arXiv:2602.00933. <https://arxiv.org/abs/2602.00933>
4. Barres, V., Dong, H., Ray, S., Si, X., & Narasimhan, K. (2025). *tau^2-Bench: Evaluating Conversational Agents in a Dual-Control Environment*. arXiv:2506.07982. <https://arxiv.org/abs/2506.07982>
5. Debenedetti, E., Zhang, J., Balunovic, M., Beurer-Kellner, L., Fischer, M., & Tramer, F. (2024). *AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents*. NeurIPS 2024 Datasets and Benchmarks Track. arXiv:2406.13352. <https://arxiv.org/abs/2406.13352>
6. Furnas, G. W. (1986). *Generalized fisheye views*. CHI 1986, 16-23. <https://doi.org/10.1145/22627.22342>
7. LangChain. (2026). *Context engineering for agents*. <https://www.langchain.com/blog/context-engineering-for-agents>
8. LangChain. (2026). *Human-in-the-loop*. <https://docs.langchain.com/oss/python/langchain/human-in-the-loop>
9. LangChain. (2026). *LangGraph overview*. <https://docs.langchain.com/oss/python/langgraph/overview>
10. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W.-t., Rocktaschel, T., Riedel, S., & Kiela, D. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS 2020. arXiv:2005.11401. <https://arxiv.org/abs/2005.11401>
11. LlamaIndex. (2025). *Context engineering: what it is, and techniques to consider*.
<https://www.llamaindex.ai/blog/context-engineering-what-it-is-and-techniques-to-consider>
12. Luo, Z., Shen, Z., Yang, W., Zhao, Z., Jwalapuram, P., Saha, A., Sahoo, D., Savarese, S., Xiong, C., & Li, J. (2025). *MCP-Universe: Benchmarking Large Language Models with Real-World Model Context Protocol Servers*. arXiv:2508.14704. <https://arxiv.org/abs/2508.14704>
13. Mireshghallah, N., Kim, H., Zhou, X., Tsvetkov, Y., Sap, M., Shokri, R., & Choi, Y. (2024). *Can LLMs Keep a Secret? Testing Privacy Implications of Language Models via Contextual Integrity Theory*. ICLR 2024. arXiv:2310.17884. <https://arxiv.org/abs/2310.17884>
14. Model Context Protocol. (2026). *Introduction*. <https://modelcontextprotocol.io/docs/getting-started/intro>
15. Model Context Protocol. (2025). *Specification, revision 2025-06-18*.
<https://modelcontextprotocol.io/specification/2025-06-18>
16. Model Context Protocol. (2025). *Tools*. <https://modelcontextprotocol.io/specification/2025-06-18/server/tools>
17. Model Context Protocol. (2025). *Resources*. <https://modelcontextprotocol.io/specification/2025-06-18/server/resources>
18. OpenAI. (2026). *Agents SDK*. <https://developers.openai.com/api/docs/guides/agents>
19. OpenAI. (2026). *Function calling*. <https://developers.openai.com/api/docs/guides/function-calling>
20. OpenAI. (2026). *Guardrails and human review*.
<https://developers.openai.com/api/docs/guides/agents/guardrails-approvals>
21. OpenAI. (2026). *Structured model outputs*. <https://developers.openai.com/api/docs/guides/structured-outputs>

22. OpenAPI Initiative. (2021). *OpenAPI Specification v3.1.0*. <https://spec.openapis.org/oas/v3.1.0.html>
23. Patil, S. G., Mao, H., Yan, F., Ji, C. C., Suresh, V., Stoica, I., & Gonzalez, J. E. (2025). *The Berkeley Function Calling Leaderboard (BFCL)*. ICML 2025, PMLR 267:48371-48392. DOI:10.5555/3780338.3782270. <https://proceedings.mlr.press/v267/patil25a.html>
24. Schick, T., Dwivedi-Yu, J., Dessi, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). *Toolformer: Language Models Can Teach Themselves to Use Tools*. NeurIPS 2023. arXiv:2302.04761. <https://arxiv.org/abs/2302.04761>
25. Trivedi, H., Khot, T., Hartmann, M., Manku, R., Dong, V., Li, E., Gupta, S., Sabharwal, A., & Balasubramanian, N. (2024). *AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents*. ACL 2024, 16022-16076. DOI:10.18653/v1/2024.acl-long.850. <https://aclanthology.org/2024.acl-long.850/>
26. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023. arXiv:2210.03629. <https://arxiv.org/abs/2210.03629>
27. Yao, S., Shinn, N., Razavi, P., & Narasimhan, K. (2024). *tau-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains*. arXiv:2406.12045. <https://arxiv.org/abs/2406.12045>